CS F342 Comp Arch — Midsem Cheatsheet (2021–2025 papers)

Paper	Question → item #
21-22	1a→5, 1b→2, 1c→3, 1d→4, 1e→29, 2a→19, 2b→30, 2c→18, 3→31
22-23	1a→1, 1b→2, 1c→4, 1d→18, 2a→23, 2b→17, 2c→24, 2d→25, 2e→26, 3→36, 4→27, 5→33, 6→28
23-24	1a→6, 1b→9, 1c→7, 1d→18, 1e→5, 1f→15, 2a→20, 2b→22, 3a→32, 3b→19, 4→39, 5a→17, 5b→34
24-25	1a→8, 1b→9, 1c→14, 1d→4, 1e→17, 1f→18, 1g→16, 2→37, 3a→21, 3b→22, 3c→38
25-26	1a→32, 1b→13, 1c→35, 2→36, 3a→9, 3b→4, 3c→10, 3d→11, 4a→14, 4b→18, 4c→16

Tag after each heading = papers where it appeared (yr-Q). Same Q again in exam → write the same answer. Opcodes: R 000000, lw 100011, sw 101011, beq 000100, addi 001000, j 000010, jal 000011. Multicycle cycles: lw 5, sw 4, R 4, addi 4, beq 3, j 3.

A. THEORY

#1 Why opcode + funct fields instead of only opcode? *22-1a*

- A 6-bit opcode can encode only 2^6=64 instructions. All R-type instrs share opcode 000000 and are told apart by the 6-bit **funct** → 64 more ALU ops without widening the 32-bit instruction.
- R-type needs no 16-bit immediate, so its low bits are free for shamt+funct; I/J types need those bits for imm/address, so they get unique opcodes. Keeps all instrs 32-bit with fields at fixed positions (regularity → simple hardware).
- Decoding becomes two-level: main decoder looks only at opcode (6 inputs); ALU decoder uses funct+ALUOp → smaller, faster control (see #10).

#2 Why is single-cycle control called a "Truth Table"? *21-1b, 22-1b*

In single-cycle MIPS every instr finishes in one clock, so the control unit is **purely combinational**: no state/memory, its outputs (RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemtoReg, ALUOp, Jump) depend *only* on the present inputs (opcode, funct). A combinational circuit is completely specified by listing outputs for every input combination = a truth table (one row per opcode), which is then implemented as SOP logic/PLA/ROM. (Multicycle control is an FSM → state table, not truth table.)

#3 How does the processor capture the side effects when instrs complete? *21-1c*

Side effects = changes to architectural state: **register file**, **data memory** and **PC**. Combinational logic computes the new values during the cycle; they are captured only at the **rising clock edge** by state elements whose write enable is asserted: RF written if RegWrite=1 (WD3→reg A3), memory written if MemWrite=1 (WD→M[A]), PC←PC' every edge (multicycle: PCEn). Intermediate results in multicycle/pipeline are held in non-architectural registers (IR, MDR, A, B, ALUOut / pipeline regs) and committed in the write-back step.

#4 <<2 behaves as <<1 — which instrs affected? *21-1d, 22-1c, 24-1d, 25-3b*

- Shift-by-2 converts a *word* offset to a *byte* offset; used only for PC-relative/jump targets.
- **beq/bne (taken)**: PCBranch = PC+4 + (SignImm<<1) = PC+4+2·imm instead of 4·imm → wrong, half-word aligned address (e.g., imm=3 → PC+4+6 not PC+4+12) → fetches garbage/misaligned instr. Not-taken branch still OK (PC+4).
- **j (and jal)**: JTA = {PC+4[31:28], addr<<1} → 26b<<1 = 27 bits only; target = half the intended address, wrong/misaligned → jumps to wrong location.
- Same in multicycle (ALUOut←PC+SignImm<<2 in Decode; jump path). **Not affected:** R-type (add, sub, and, or, slt), lw, sw, addi — they don't use the shifters (lw/sw offsets are already byte offsets).

#5 Stuck control signals (single-cycle) *21-1a, 23-1e*

Signal	Malfunction & why
MemtoReg=1	**R-type (add,sub,and,or,slt) & addi**: write ReadData (=M[ALUResult]) to reg instead of ALUResult. lw OK; sw, beq, j OK (RegWrite=0).
MemtoReg stuck-at-0	**lw only**: writes ALUResult (the address RF[rs]+imm) into rt instead of loaded data. All others OK.
ALUOp 0=1	ALUOp 00→01 ⇒ ALU **subtracts: lw, sw** (address = rs−imm), **addi** (rs−imm) wrong. beq (01) OK. R-type: 10→11, ALU decoder looks at funct when ALUOp1=1 (11 treated as 1X) → OK (if 11 were undefined, R-type would also fail). J OK.
RegDst =0	**R-type**: result written to rt (20:16) instead of rd (15:11) → wrong reg & corrupts rt. lw/addi OK (use rt); sw, beq, j OK (no write).

#6 Can rt be the dest for R-type (and rd for others)? *23-1a*

Not advisable / No. MIPS reads registers from fixed fields: Read reg1 = rs (25:21), Read reg2 = rt (20:16) for *all* instrs, so the RF can be read **in parallel with decoding**. sw and beq must read rt as a source; if R-type wrote rt, its 2nd source would be in rd, so a mux (controlled by decoder) would be needed on **Read register 2** → RF read starts only after control decode → longer critical path (t_{ctrl}+t_{mux}). Vice-versa (I-type writing rd) is impossible: bits 15:11 are part of the 16-bit immediate in lw/addi. Hence dest mux (RegDst) is on the write side: rt for I-type, rd for R-type.

#7 Do we need opcode in a special-purpose processor? *23-1c*

No. A single-purpose processor executes one fixed algorithm; the algorithm is hard-coded in the controller FSM which directly sequences the datapath control signals each state. There is no program/instruction memory, no fetch and no decode, so no opcode. Only status inputs (e.g., comparator outputs) feed the controller. Opcode is needed only in general-purpose (programmable) processors.

#8 Single-cycle vs multi-cycle control unit *24-1a*

Single-cycle	Multi-cycle
Combinational (truth table): signals = f(opcode,funct)	Sequential FSM: state reg + next-state logic + output logic; signals = f(state) (Moore), next state = f(state, opcode)
Signals constant for whole (long) cycle; CPI=1	Signals change every state; CPI 3–5 cycle; CPI=1
Signals: RegDst, RegWrite, ALUSrc, Branch, MemWrite, MemtoReg, ALUOp, Jump	Adds IorD, IRWrite, PCWrite, ALUSrcA, ALUSrcB[1:0], PCSrc[1:0]; enables for PC/IR
Separate I/D memories & adders → no sequencing	Shared memory & ALU reused across states
Only hardwired	Hardwired FSM or microprogrammed (#9)
Control Table: Jump, RegDst, RegWrite, ALUSrc, Branch, ALUOp, MemRead, MemWrite, MemtoReg (R3)	States T0 IF, T1 ID, T2 EXE(lw/sw/addi), T3 MEM(lw), T4 WB(lw), T5 MEM(sw), T6 EXE(R), T7 WB(R), T8 EXE(beq), T9 WB(addi), T10 J (R7).

#9 How are states generated: hardwired vs microprogrammed? *23-1b, 24-1b, 25-3a*

Hardwired (prof): states generated by a **3-bit counter** (CLK, INCR, CLR; not loadable) + **state decoder SDCD 3:8** → T0…T7, common to all instrs (T0 IF, T1 ID, T2 EXE, T3 MEM/WB, T4 WB); **IDCD 6:64** decodes opcode; each control = SOP of T_i·instr (e.g. MemWrite = T3·SW, Jump = T2·J); INCR = T0+T1+T2·LW+T3·LW+T2·SW…, CLR = T4·LW+T3·SW+… returns to T0 (see R7b). (Classic FSM form: 4-bit state register + next-state logic S' = f(S, opcode).)
Microprogrammed (prof): each state = one **microinstruction** [M | CF | NA] in **control memory**; state = address held in **CMAR** (µPC). Next state: **NA** field of current word (Mode=0) or, in Decode (Mode=1), the **external address** from the decoding table indexed by IR opcode (LW→2, SW→5, ADD→7, ADDI→9, J→11, BEQ→12), selected by a MUX into CMAR; last word of each instr has NA=0 (Fetch) (see R7, R7c). Control signals are read directly from the µinstr bits. Easy to modify, slower; hardwired faster.

#10 Why two-level control (main decoder + ALU decoder)? *25-3c*

Main decoder: opcode(6) → RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemtoReg, Jump, **ALUOp[1:0]**. ALU decoder: ALUOp(2)+funct(6) → ALUControl. Benefits: (1) **smaller logic**: a monolithic controller would have 12 inputs (2^{12}=4096 rows) vs 2^6+2^8; (2) faster & simpler to design/verify; (3) **modular**: adding an R-type instr only changes the ALU decoder; ALU decoder reused in multicycle & pipeline; (4) funct only matters for R-type, so it isn't routed into the main decoder.

#11 PC enable: single vs multi-cycle? *25-3d*

Yes. Single-cycle: PC has **no enable** — every instruction completes in 1 cycle, so PC←PC' at every clock edge. Multicycle: an instr takes 3–5 cycles and the PC (memory address, PC+4, branch target computations) must hold its value during intermediate states, so PC has an enable: **PCEn = PCWrite + Branch·Zero** (+ **Jump** in prof's datapath: OR gate of PCWrite, Jump, Branch·Zero). PCWrite=1 only in Fetch (PC←PC+4); Branch·Zero in beq state; Jump in J state.

#12 Why can't single-cycle use one memory for fetch + data? *25-1a*

A single-ported memory has one address port and one read/write data port → only **one access per cycle**. Single-cycle lw/sw need instruction fetch AND data read/write in the **same** cycle → structural hazard (address port conflict). **Fix:** separate **instruction memory and data memory** (Harvard; split I-cache/D-cache) — or a dual-ported memory; (multicycle instead reuses one memory in different cycles with the IorD mux).

#13 Illegal control-signal combination *25-1b*

e.g. **RegWrite=1 with MemWrite=1** (during sw): memory is written correctly but the RF also writes Result (ALUResult = address, or garbage ReadData) into register rt/rd (whatever RegDst selects) → the source register (rt) being stored gets **corrupted**. Others: Branch=1 & Jump=1 (two PC sources, Jump mux overrides → wrong PC); MemtoReg=1 with MemRead=0 (writes undefined data).

#14 Methods of generating control signals in pipeline *24-1c, 25-4a*

Problem (slides 30–31): 10 control lines are needed in later stages, but while instr i is in EX, instr i+1 is being decoded → if controls are not stored, i+1's controls overwrite i's (erroneous signals).

1. **Strategy-1 (slide 27): pass the instruction, decode in every stage**: the instruction (opcode/funct) travels in IF/ID, ID/EX, EX/MEM, MEM/WB and each stage has its own control unit generating only its signals. *Adv*: narrow stage regs (just instr bits), simple per-stage control. *Disadv*: 4 duplicated decoders (area), decode delay added inside each stage's critical path.
2. **Strategy-2 (slide 28, P&H): decode once in ID, pass control bits**: same control unit as single-cycle (truth table) generates all 10 lines in ID; stage registers are extended with fields EX/M/WB: ID/EX 9 bits (RegDst, ALUOp[1:0], ALUSrc, Branch, MemRead, MemWrite, RegWrite, MemtoReg), EX/MEM 5 bits, MEM/WB 2 bits; each stage consumes its group (EX: RegDst, ALUOp, ALUSrc; MEM: Branch, MemRead, MemWrite; WB: RegWrite, MemtoReg; Jump used in IF). *Adv*: one decoder, reuses single-cycle CU, decode delay off the later stages. *Disadv*: wider pipeline regs (more flip-flops/power).
3. **Microprogrammed/RCM**: flexible, easy to change; slower, big ROM. Hazard unit separately generates Stall, Flush, Forward signals.

#15 Can forwarded data be taken before the stage (pipeline) registers? (R-type→R-type) *23-1f*

No. I1 computes its result in EX in cycle c; I2 needs it in its EX in cycle c+1. In cycle c+1 the wire before EX/MEM carries I2's *own* ALU output (I1's value is now in EX/MEM) → wrong data / combinational loop. Taking it in cycle c would chain ALU → forward mux → ID path in one cycle (longer clock). The pipeline register holds I1's result stable for the whole next cycle, so forward from **EX/MEM** (ALUOutM) and **MEM/WB** (ResultW) registers: ForwardAE=10 if (rsE≠0 & rsE==WriteRegM & RegWriteM), 01 if from WB.

#16 Setup & hold time; derive clock constraints *24-1g, 25-4c*

Setup t_{setup}: min time D must be stable *before* the clock edge. **Hold t_{hold}**: min time D must stay stable *after* the edge. Path R1 → combinational logic → R2: **Setup:** data launched at edge 0 arrives at R2 latest at t_{pcq}+t_{pd}; must arrive t_{setup} before next edge: $T_c \geq t_{pcq}+t_{pd}+t_{setup} \Rightarrow t_{pd} \leq T_c - (t_{pcq}+t_{setup})$ (sequencing overhead).
Hold: new data launched at the same edge must not reach R2 before t_{hold}: earliest arrival t_{ccq}+t_{cd}: $t_{ccq}+t_{cd} \geq t_{hold}$ (independent of T_c; fix by adding buffers). With clock skew: $T_c \geq t_{pcq}+t_{pd}+t_{setup}+t_{skew}$; $t_{ccq}+t_{cd} \geq t_{hold}+t_{skew}$. (pcq/pd = propagation = max; ccq/cd = contamination = min.)

#17 128 registers & instruction set ×64 — effect on fields *22-2b, 23-5a, 24-1e*

Reg fields: $\log_2$128 = **7 bits** (was 5). 4× instrs → opcode **6→8 bits** (and funct 6→8 if R-types also ×4).
R-type: op 8 + rs 7 + rt 7 + rd 7 + shamt 5 + funct 8 = **42 bits** (+10). **I-type**: op 8 + rs 7 + rt 7 + imm 16 = **38 bits** (+6).
If instruction must stay 32-bit: I-type imm shrinks to 32−8−14 = **10 bits** (smaller constants/branch range); R-type has only 32−8−21 = **3 bits** left for shamt+funct → must drop shamt/shrink funct or use more opcodes. Effects: bigger/slower RF, bigger decoder, fewer bits for constants.

B. NUMERICALS / TRACING

#18 5-stage pipeline: regs written/read in the 5th cycle *21-2c, 22-1d, 23-1d, 24-1f, 25-4b*

Cycle 5: I1 in **WB**, I2 MEM, I3 EX, **I4 in ID**, I5 IF. RF is **written in 1st half** of cycle (by I1) and **read in 2nd half** (by I4, which reads its rs & rt fields). (Prof: RF written @posedge, stage registers @negedge ⇒ write completes before I4's read is latched.)

Program (I1 … I5)	Written	Read (by I4)
21: or R2,R4,R5 / sw R5,72(R0) / sub R0,R1,R2 / **lw R3,15(R1)** / addi	R2 (or)	R1 (base; R3 on RD2 unused)
22: or $t2,$t4,$t5 / sub / sw / **addi $t1,$t1,50** / lw	$t2	$t1
23,24: or $t2,$t4,$t5 / sw / addi / **lw $t3,15($t1)** / sub	$t2	$t1 ($t3 unused)
25: addi $t1,$t1,50 / lw / or / **sw $t5,72($t0)** / sub	$t1	$t0, $t5

```
     c1 c2 c3 c4  c5
I1     IF ID EX MEM WB  ← writes (1st half)
I2        IF ID EX  MEM
I3           IF ID  EX
I4              IF  ID  ← reads (2nd half)
I5                  IF
```

#19 Cycles & CPI of loop program (single/multi-cycle) *21-2a, 23-3b*

R1=0, R2=10; loop adds 2 to R1 → beq evaluated at R1=0,2,4,6,8 (not taken) and 10 (taken) ⇒ beq ×6, loop body (3 addi + j) ×5.
Instr count = 2 addi + 6 beq + 15 addi + 5 j = **28**.
Multicycle (addi 4, beq 3, j 3): 17×4 + 6×3 + 5×3 = 68+18+15 = **101 cycles**; CPI = 101/28 = **3.61**.
Single-cycle: 28 cycles, CPI = **1** (but long clock).

#20 Single-cycle fetches 0x0800248C: values in ALU ctrl, muxes, ALU, adders *23-2a*

0000 10|00 0000 0000 0010 0100 1000 1100 → op=000010 = **j** (J-type), addr26 = 0x000248C. (Field view: rs=0, rt=0, rd=00100, shamt=10010, funct=001100, imm=0x248C.) Assume PC=0x00400000, $0=0.
Control (j, prof's table): Jump=1, RegWrite=0, MemRead=0, MemWrite=0, RegDst/ALUSrc/Branch/MemtoReg = X, ALUOp=XX.

Unit	Inputs → Output
A1 (PC+4)	0x00400000, 4 → **0x00400004**
SignExt / <<2	0x248C → 0x0000248C → 0x00009230
A2 (branch)	0x00400004 + 0x00009230 → 0x00409234
RF	A1=rs=0, A2=rt=0 → RD1=0, RD2=0
M2 RegDst	in0 = 00000 (rt), in1 = 00100 (rd) → X (RegWrite=0)
M3 ALUSrc	in0 = RD2 = 0, in1 = 0x0000248C → X
ALU decoder	ALUOp=XX, funct=001100 → ALUControl = XXXX (if ALUOp=00: 0010 add)
ALU	SrcA=RD1=0, SrcB=M3 out → result unused (X); Zero=X
Data mem	MemWrite=0 → not written; RD unused
M5 MemtoReg	in0 ALUResult, in1 ReadData → X (not written)
M4 PCSrc	Branch·Zero=0 → in0 = **0x00400004**
M1 Jump	in0 = 0x00400004, in1 = {PC+4[31:28]=0000, 0x000248C, 00} = 0x00009230; Jump=1 → **PC' = 0x00009230**

#21 Multicycle fetches 0x1000248C — values per state (M0–M6 of given fig) *24-3a*

0001 00|00 000|0 0000|0010 0100 1000 1100 → op=000100 = **beq $0,$0,0x248C** (rs=rt=0, imm=0x248C; rd bits=00100). Assume PC=0x00400000. 3 states:

State	Signals	Values
S0 Fetch	IorD=0, IRWrite=1, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, PCWrite=1, Jump=0	M0: Adr=PC=0x00400000: Mem RD=0x1000248C→IR. M4=PC, M5=4, ALUCtrl=0010 → ALU=0x00400004; M6→M1 → PC=0x00400004 (also ALUOut)
S1 Decode	ALUSrcA=0, ALUSrcB=11, ALUOp=00	RF A1=A2=0 → A=0, B=0; SignImm=0x0000248C, <<2=0x00009230; M4=0x00400004, M5=0x00009230 → ALU=0x00409234 → ALUOut. M2 in: 0/4 (unused)
S8 Branch	ALUSrcA=1, ALUSrcB=00, ALUOp=01, Branch=1, PCSrc=01	M4=A=0, M5=B=0, ALUCtrl=0110 → 0−0=0, **Zero=1** ⇒ PCEn=Branch·Zero=1; M6=ALUOut → M1(Jump=0) → **PC=0x00409234** (branch taken)

M3 (MemtoReg) unused: RegWrite=0 throughout.

#22 Latencies (Table-1) & minimum clock period *23-2b*

mem 250, RF 150, mux 25, ALU 200, adder 150, gate 5, clk-q 30, setup 20, sext 50, control 50.

Instr	Critical path	ps
R-type	t_{pcq}+t_{mem}+t_{RF}+t_{mux}(ALUSrc)+t_{ALU}+t_{mux}(MemtoReg)+t_{setup} = 30+250+150+25+200+25+20	**700**
LW	30+250+max(RF 150, sext+mux 75)+200+250(Dmem)+25+20	**925**
SW	30+250+150+200+250 (Dmem write)	**880**
BEQ	30+250+150+25+200+5(AND)+25(M4)+25(M1)+20(PC setup) [adder path 30+250+50+150=480 not critical]	**730**

Control (30+250+50=330 ps) is ready before RF data (430) → not on critical path.
$T_{c,min}$ **= max = 925 ps (lw)** → f = 1.08 GHz.

#23 Bytes M3=00100000, M2=01000000, M1=00001000, M0=00000001 → instr *22-2a*

MIPS big-endian (M0 = MSB): 0000 0001 0000 1000 0100 0000 0010 0000 = 0x01084020 → 000000|01000|01000|01000|00000|100000 ⇒ **R-type: add $t0, $t0, $t0** (op 0, rs=rt=rd=8, shamt 0, funct 0x20).
(If little-endian, M3 = MSB: 0x20400801 = 001000|00010|00000|0000100000000001 ⇒ I-type addi $zero, $v0, 2049.)

#24 PC = 0x30000000: range reachable by beq *22-2c*

Target = PC+4 + SignExt(imm)·4, imm 16-bit ∈ [-2^{15}, 2^{15}−1]. PC+4 = 0x30000004.

Min = 0x30000004 – 2^{17} = 0x30000004 – 0x20000 = **0x2FFE0004**

Max = 0x30000004 + (2^{15}−1)·4 = 0x30000004 + 0x1FFFC = **0x30020000** = ±128 KB (−32768 … +32767 instructions from PC+4), word aligned.

#25 Horizontal µprogram control-memory width *22-2d*

Horizontal: one bit per control line → control field = **11 bits**. Condition select for 8 status bits = $\log_2$8 = **3 bits** (MUX select). Next-address for 128 words: $\log_2$128 = **7 bits**.
Width = 11 + 3 + 7 = **21 bits**; control memory = 128 × 21 bits. (If status were also one-hot: 11+8+7 = 26. Prof's format [M | CF | NA] adds a 1-bit mode/dispatch field → 22.)

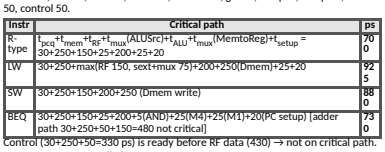
#26 Translate MIPS to C (D base $s6, m=$s0) *22-2e*

```
addi $t0,$s6,4    # t0 = &D[1]
add  $t1,$s6,$0   # t1 = &D[0]
sw   $t1,0($t0)   # D[1] = &D[0]
lw   $t0,0($t0)   # t0 = D[1] (= &D[0])
add  $s0,$t1,$t0  # m = &D[0] + D[1]
add  $s0,$s0,$0   # m = m + D[1]
```

C: `D[1] = (int)&D[0]; m = (int)&D[0] + D[1] + D[1];` ⇒ m = 3·(base address of D).

#27 ld/sd (no offset) in single-cycle *22-4*

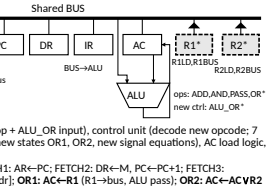
(a) Old lw: 30+250+150+200+250+25+20 = 925 ps. New ld needs no ALU: t_{pcq}+t_{mem}+t_{RFread}+t_{mux}+$t_{RFwrite}$ = 30+250+150+250+25+20 = **725**. R-type = 30+250+150+25+200+25+20 = 700; beq = 700. ⇒ T_c **= 725 ps** (ld still longest).
(b) Each lw/sw now needs an extra addi for the address: N' = N(1+0.26+0.12) = 1.38N. Old: Nx925 = 925N. New: 1.38Nx725 = 1000.5N ⇒ **slower**; 1000.5/925 = 1.082 → ≈ **8.2% slower**.
(c) The fraction f of loads/stores (that need an extra address instruction) vs the cycle-time gain: faster iff (1+f)·725 < 925 ⇒ **f < 27.6%**. (Here f = 38% → slower.)
(d) Data-memory address taken directly from RD1:



C. DESIGN / DATAPATH MODIFICATIONS

#29 Simple CPU (shared bus): add OR R1,R2 (AC ← R1 ∨ R2) *21-1e*

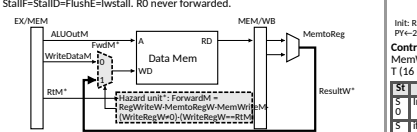
(1) Min components: registers **R1, R2** (if absent) with load (R1LD, R2LD) and **tri-state buffers** onto the bus (R1BUS, R2BUS); ALU gets an OR function (ALU_OR). Single bus ⇒ two operands can't travel together, so AC is used as temp: AC←R1, then AC←AC∨R2.



(2) Modified: ALU (OR op + ALU_OR input), control unit (decode new opcode; 7 instrs ⇒ 3-bit opcode, new states OR1, OR2, new signal equations), AC load logic, bus (R1, R2 drivers).
(3) States / µops: FETCH1: AR←PC; FETCH2: DR←M, PC←PC+1; FETCH3: IR←DR[op], AR←DR[addr]; **OR1: AC←R1** (R1→bus, ALU pass); **OR2: AC←AC∨R2** → FETCH1.
OR1 = I_{OR}·T3, OR2 = I_{OR}·T4 (I_{OR} = opcode decoder output). **R1BUS = OR1; R2BUS = OR2; ALU_OR = OR2; ALU_PASS = LD2 + OR1; ACLD = LD2 + ADD2 + AND2 + OR1 + OR2** (OR2 (old terms + new); CLR seq. counter after OR2.

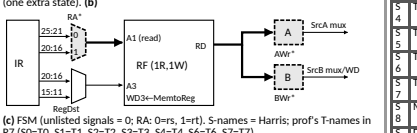
#30 Pipeline: hazard LW R1,off(R1) → SW R1,off(RR) *21-2b*

sw needs R1 only in MEM (as write data); lw's data is ready at end of its MEM ⇒ forward **MEM/WB → MEM** (no stall). Add RtM (5 b) to EX/MEM and a mux before Dmem WD.
Detect (ForwardM=1): RegWriteW · MemtoRegW · MemWriteM · (WriteRegW ≠ 0) · (WriteRegW == RtM). WriteDataM' = ForwardM ? ResultW : WriteDataM.
If base RK == RI (address needs the loaded value in EX) ⇒ load-use **stall** 1 cycle: lwstall = MemtoRegE · (RsD == RtE); modify old lwstall so sw data reg doesn't stall: Stall = MemtoRegE·((RsD==RtE) + (RtD==RtE)) ~MemWriteD);
StallF=StallD=FlushE=lwstall. R0 never forwarded.



#31 Multicycle with 1-read-port RF *21-3*

(a) Add a **2:1 MUX, 5-bit** (inputs IR[25:21]=rs and IR[20:16]=rt, 1 select "RA", 5-bit output) on the single Read-register address. The single RD (32 b) feeds both A and B, so A and B get **write enables** (AWr, BWr). Modified: RF, A, B, controller (one extra state). **(b)**



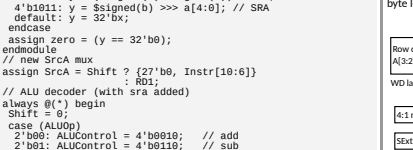
(c) FSM (unlisted signals = 0; RA: 0=rs, 1=rt). S-names = Harris; prof's T-names in R7 (S0=T0, S1=T1, S2=T2, S3=T3, S4=T4, S6=T6, S7=T7).

State	µop	Signals	Next
S0 Fetch	IR←M[PC]; PC←PC+4	IorD=0, IRWrite, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, PCWrite	S1
S1 Decode-1	A←RF[rs]; ALUOutCalcW←PC+SImm <<2	RA=0, AWr=1, ALUSrcA=0, ALUSrcB=11, ALUOp=00	R→S1b; lw→S2
S1b Decode-2	B←RF[rt]	RA=1, BWr=1	S6
S6 Execute	ALUOut←A op B	ALUSrcA=1, ALUSrcB=00, ALUOp=10	S7
S7 ALU WB	RF[rd]←ALUOut	RegDst=1, MemtoReg=0, RegWrite	S0
S2 MemAdr	ALUOut←A+SImm	ALUSrcA=1, ALUSrcB=10, ALUOp=00 (sw: also RA=1, BWr)	S3
S3 MemRead	MDR←M[ALUOut]	IorD=1	S4
S4 Mem WB	RF[rt]←MDR	RegDst=0, MemtoReg=1, RegWrite	S0

Optimized: lw needs only rs ⇒ skips S1b (still 5 cycles); R-type 5 cycles (+1). sw reads rt during S2 (port free).

#32 Add sra $rd,$rt,shamt to single-cycle (+Verilog) *23-3a, 24-3b*

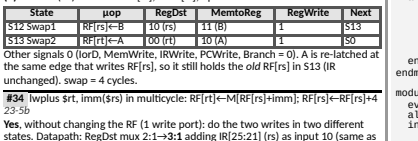
rd ← rt >>> shamt (arith). Shift amount = Instr[10:6] must reach ALU; add a **2:1 mux on SrcA** (RD1 / zero-ext shamt) with new signal **Shift**; ALU gets SRA op (ALUControl=1011): Result = $signed(SrcB) >>> SrcA[4:0]. Main decoder: R-type row unchanged (RegWrite=1, RegDst=1, ALUSrc=0, MemtoReg=0, ALUOp=10). ALU decoder: ALUOp=10 & funct=000011 (table: 100011) → ALUControl=1011, Shift=1. Other instrs: Shift=0 → unaffected.




```
module alu(input [31:0] a, b, input [3:0] f,
           output reg [31:0] y, output zero);
 always @(*) case (f)
  4'b0000: y = a & b;
  4'b0001: y = a | b;
  4'b0010: y = a + b;
  4'b0110: y = a - b;
  4'b0111: y = ($signed(a)<$signed(b)) ? 1:0;
  4'b1011: y = $signed(b) >>> a[4:0];  // SRA
  default: y = 32'bx;
 endcase
 assign zero = (y == 32'b0);
endmodule
// new SrcA mux
assign SrcA = Shift ? {27'b0, Instr[10:6]}
                    : RD1;
// ALU decoder (with sra added)
always @(*) begin
 Shift = 0;
 case (ALUOp)
  2'b00: ALUControl = 4'b0010;   // add
  2'b01: ALUControl = 4'b0110;   // sub
  default: case (Funct)
   6'b100000: ALUControl = 4'b0010;
   6'b100010: ALUControl = 4'b0110;
   6'b100100: ALUControl = 4'b0000;
   6'b100101: ALUControl = 4'b0001;
   6'b101010: ALUControl = 4'b0111;
   6'b000011: begin                  // sra
     ALUControl = 4'b1011; Shift = 1; end
   default:   ALUControl = 4'bxxxx;
  endcase
 endcase
end
```

#33 Add swap rs,rt to multicycle [Reg[rs]=Reg[rt]; Reg[rt]=Reg[rs]] *22-5*

One write port ⇒ two write-back states. **(a)** RegDst mux 2:1→**3:1** (add rs = IR[25:21]); MemtoReg mux 2:1→**4:1** (add A and B register outputs). RegDst, MemtoReg become 2-bit.



(b) Decode (S1) latches A=RF[rs], B=RF[rt]; op=SWAP → S12.

State	µop	RegDst	MemtoReg	RegWrite	Next
S12 Swap1	RF[rs]←B	10 (rs)	11 (B)	1	S13
S13 Swap2	RF[rt]←A	00 (rt)	10 (A)	1	S0

Other signals 0 (IorD, MemWrite, IRWrite, PCWrite, Branch = 0). A is re-latched at the same edge that writes RF[rs], so it still holds the *old* RF[rs] in S13 (IR unchanged). swap = 4 cycles.

#34 lwplus $rt, imm($rs) in multicycle: RF[rt]←M[RF[rs]+imm]; RF[rs]←RF[rs]+4 *23-5b*

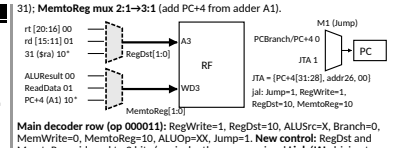
Yes, without changing the RF (1 write port): do the two writes in two different states. Datapath: RegDst mux 2:1→**3:1** adding IR[25:21] (rs) as input 10 (same as #33 left mux). ALU computes rs+4 using the existing "4" input of ALUSrcB while **memory** is read.

State	µop	Signals (rest 0)	Next
S0, S1	Fetch, Decode (std)	A←RF[rs]	S2
S2 MemAdr	ALUOut←A+SImm	ALUSrcA=1, ALUSrcB=10, ALUOp=00	S3
S3 MemRd+Inc	MDR←M[ALUOut]; ALUOut←A+4	IorD=1, ALUSrcA=1, ALUSrcB=01, ALUOp=00	S4
S4 WB1	RF[rt]←MDR	RegDst=00, MemtoReg=1, RegWrite=1	S5*
S5* WB2	RF[rs]←ALUOut	RegDst=10, MemtoReg=0, RegWrite=1	S0

In S3 memory uses the old ALUOut (address) during the cycle; ALUOut is overwritten with A+4 only at the end edge. lwplus = 6 cycles.

#35 Add jal label to single-cycle *25-1c*

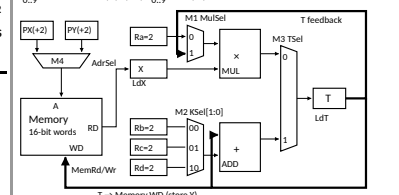
jal: PC ← JTA = {PC+4[31:28], addr26, 00} (same as j via Jump mux M1) **and** $ra($31) ← PC+4. Modifications: **RegDst mux 2:1→3:1** (add constant 5'b11111 = 31); **MemtoReg mux 2:1→3:1** (add PC+4 from adder A1).



Main decoder row (op 000011): RegWrite=1, RegDst=10, ALUSrc=X, Branch=0, MemWrite=0, MemtoReg=10, ALUOp=XX, Jump=1. **New control:** RegDst and MemtoReg widened to 2 bits (equivalently one new signal **Link/JAL** driving two extra 2:1 muxes: WriteReg = JAL ? 31 : old; Result = JAL ? PC+4 : old). Existing instrs use 00/01 → unaffected. Return later with jr $ra.

#36 16-bit multi-cycle single-purpose processor: Y_i=ai^3+bi^2+cY_i+d, a=b=c=d=2, 0≤i<10 *22-3, 25-2*

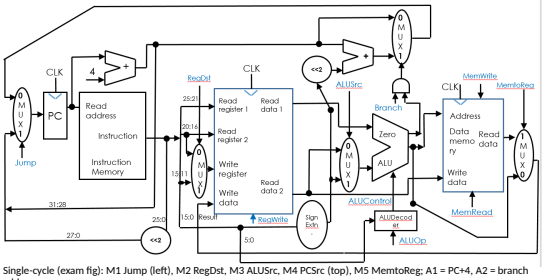
Horner: **Y = ((a·X + b)·X + c)·X + d** → 3 multiplies + 3 adds using 1 multiplier + 1 adder over several cycles. 16-bit data, byte-addressable ⇒ pointers step by **2**. $X_{0..9}$ at addresses 0,2,…,18; $Y_{0..9}$ at 20,…,38.



Control signals (13): Init, LdX, LdT, MulSel, KSel[1:0] (2), TSel, AdrSel, MemRd, MemWr, IncX, IncY, Incl. Status in: Eq10; output: Stop. Storage: Ra, Rb, Rc, Rd, X, T (16 b), PX, PY (16 b), i (4 b).

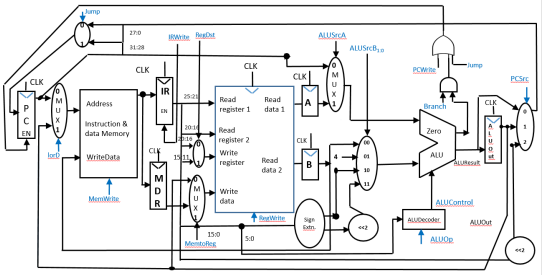
St	Operation	Asserted (rest 0)	Next
S0	Init: Ra..Rd←-2, PX←0, PY←20, i←0	Init	S1
S1	if Eq10 → Stop; else X←M[PX], PX←PX+2	AdrSel=0, MemRd, LdX, IncX	Eq10? S9:S2
S2	T←-a·X	MulSel=0, TSel=0, LdT	S3
S3	T←T+b	KSel=00, TSel=1, Ld

Single-Cycled MIPS microprocessor's datapath

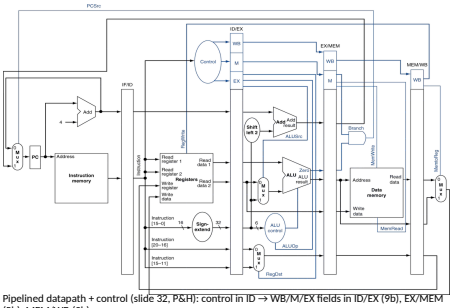


Single-cycle (exam fig): M1 Jump (left), M2 RegDst, M3 ALUSrc, M4 PCSrc (top), M5 MemToReg; A1 = PC+4, A2 = branch adder.

Multi-Cycled MIPS microprocessor's datapath



Multicycle (exam fig): M0 IorD, M1 Jump, M2 RegDst, M3 MemtoReg, M4 ALUSrcA, M5 ALUSrcB, M6 PCSrc; non-arch regs IR, MDR, A, B, ALUOut.



Pipelined datapath + control (slide 32, P&H): control in ID → WB/M/EX fields in ID/EX [9b], EX/MEM (5b), MEM/WB (2b).

D. REFERENCE — slides + Harris/P&H

R1 Instruction formats & 5 addressing modes

Type	31:26	25:21	20:16	15:11	10:6	5:0
R	op(6)=0	rs	rt	rd	shamt	funct
I	op	rs	rt	imm16 (offset)		
J	op	address26				

1. **Register**: all operands in regs — add $s0,$s1,$s2
2. **Immediate**: 16-bit constant in instr (sign-ext) — addi $s0,$s1,5
3. **Base (displacement)**: addr = RF[rs] + SignExt(imm) — lw $t0,8($s1)
4. **PC-relative**: target = PC+4 + SignExt(imm)·4 — beq/bne
5. **Pseudo-direct**: target = {PC+4[31:28], addr26, 00} — j, jal

Design principles: simplicity favours regularity; make the common case fast; smaller is faster; good design demands good compromises. MIPS good for pipelining (slide 34): all instrs 32-bit, few addressing modes & formats, memory operands only in lw/sw, operands aligned (x86 1–15 bytes → hard).

R2 Opcodes / funct (hex)

Instr	op	funct	Operation
add	000000	100000 (20)	rd=rs+rt
sub	000000	100010 (22)	rd=rs−rt
and / or	000000	100100 / 100101	rd=rs&rt / rs|rt
slt	000000	101010 (2A)	rd=(rs<rt)
sll / srl / sra	000000	000000 / 000010 / 000011	rd=rt<<,>>,>>> shamt
jr	000000	001000 (08)	PC=rs
lw / sw	100011 (23) / 101011 (2B)	—	rt=M[rs+imm] / M[rs+imm]=rt
beq / bne	000100 (04) / 000101 (05)	—	If(rs==/!=rt) PC=PC+4+imm·4
addi / slti	001000 (08) / 001010	—	rt=rs+SignExt(imm)
andi / ori / lui	001100 / 001101 / 001111	—	ZeroExt imm; lui rt=imm<<16
j / jal	000010 / 000011	—	PC=ΠA; jal: $ra=PC+4

Opcode warning: prof's slides (M1-a 39, M1-a 66) & 2024 paper table use **LW = 100010, SW = 100011**; real MIPS & 2023 paper use LW 100011, SW 101011 — use whatever table the question gives. Shifts (M1-a 82–84): sll $t0,$s1,4 = 0x00114100; srl $s2,$s1,4 = 0x00119102; sra $s3,$s1,4 = 0x00119903 (rs=0, rt=src, shamt); sllv/srlv/srav (funct 4/6/7) use rs as amount: sllv $s3,$s1,$s2 = 0x02519804. $s1=0xF30002A8 → sll 4: 0x30002A80, srl 4: 0x0F30002A, sra 4: 0xFF30002A. Loads/stores: lb rt,imm(rs): RF[rt]=SExt(M[RF[rs]+SExt(imm)](7:0)); lbu zero-ext; lh/lhu 15:0; sb M[..](7:0)=RF[rt](7:0); sh 15:0.

Regs: $0 zero | $1 $at | $2–3 $v0–1 | $4–7 $a0–3 | $8–15 $t0–7 | $16–23 $s0–7 | $24–25 $t8–9 | $26–27 $k0–1 | $28 $gp | $29 $sp | $30 $fp | $31 $ra.

R3 Main decoder truth table (single-cycle; prof slide 25)

Instr	op	Jmp	RDst	RWr	ASrc	Br	ALUOp	MRd	MWr	M2R
R-type	000000	0	1	1	0	0	10	0	0	0
lw	100011	0	0	1	1	0	00	1	0	1
sw	101011	0	x	0	1	0	00	0	1	x
addi	001000	0	0	1	1	0	00	0	0	0
beq	000100	0	x	0	0	1	01	0	0	x
j	000010	1	x	0	x	x	xx	0	0	x

(RDst=RegDst, RWr=RegWrite, ASrc=ALUSrc, Br=Branch, MRd/MWr=MemRead/Write, M2R=MemtoReg.) RegDst 0=rt,1=rd; ALUSrc 0=RD2,1=SignImm; MemtoReg 0=ALUResult,1=ReadData; PCSrc = Branch·Zero (bne: Branch·~Zero); Jump selects JTA. (Pipelined table slide 29 shows sw MemRead 1 — use 0.)

Harris controls[8:0] = {RegWrite,RegDst,ALUSrc,Branch,MemWrite,MemtoReg,Jump,ALUOp}: R 11000010, lw 101001000, sw 001010000, beq 000100001, addi 101000000, j 000000100.

R4 ALU decoder & ALU control lines / functions

ALUOp	funct	ALUCtrl 4b (prof/P&H)	3b (Harris)	action
00	x	0010	010	add (lw, sw, addi)
01	x	0110	110	sub (beq/bne)
10	100000	0010	010	add
10	100010	0110	110	sub
10	100100	0000	000	AND
10	100101	0001	001	OR
10	101010	0111	111	slt
10	000011	1011	—	sra (exam table)

4-bit ALU control lines = Ainvert | Bnegate | Operation[1:0] (00 AND, 01 OR, 10 add, 11 slt/less): 0000 AND, 0001 OR, 0010 add, 0110 sub (Bnegate+add, carry-in 1), 0111 slt, 1100 NOR. **Harris 3-bit F:** 000 A&B, 001 A|B, 010 A+B, 011 unused, 100 A&~B, 101 A|~B, 110 A−B, 111 SLT. F2 inverts B (B' = ~B, Cin=1 → subtract); SLT = sign bit of A−B, zero-extended. **Zero** = NOR of all result bits (used by beq).

R5 Single-cycle: Tc of every instruction (Harris/slide delays: t_{pcq} 30, t_{setup} 20, t_{mux} 25, t_{ALU} 200, t_{mem} 250, t_{RFread} 150, $t_{RFsetup}$ 20, $t_{RFwrite}$ 100)

Instr	Critical path	ps
lw	$t_{pcq}+t_{mem}+\max(t_{RFread}, t_{sext}+t_{mux})+t_{ALU}+t_{mem}+t_{mux}+t_{RFsetup}$ = 30+250+150+200+250+25+20	925
sw	$t_{pcq}+t_{mem}+t_{RFread}+t_{ALU}+t_{mem}$(write) = 30+250+150+200+250	880
R-type	$t_{pcq}+t_{mem}+t_{RFread}+t_{mux}+t_{ALU}+t_{mux}+t_{RFsetup}$ = 30+250+150+250+200+25+20	700
addi	$t_{pcq}+t_{mem}+t_{RFread}+t_{ALU}+t_{mux}+t_{RFsetup}$	675
beq	$t_{pcq}+t_{mem}+t_{RFread}+t_{mux}+t_{ALU}+t_{AND}+t_{mux}$(PCSrc)$+t_{mux}$(Jump)$+t_{setup}$	~725+t AND
j	$t_{pcq}+t_{mem}+t_{mux}+t_{setup}$ = 30+250+25+20	325

Prof (M-1 75): $T_c = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFsetup}$ = 30+2(250)+150+200+25+20 = **925 ps**. Single-cycle: IM, RF, DM read combinationally; all writes on rising clock edge; whole instr in one cycle; simple CU (no next state).

Instr	PC	IMM	ADD_PC	RF	EXTN	ALU	<<2	ADD_B	DMM	RFwr	Total
ADD	√	√	√	√	x	√	x	x	x	√	6
BNE	√	√	√	√	√	√	√	√	x	x	8
J	√	√	√	x	x	x	x	x	x	x	3
SW	√	√	√	√	√	√	x	x	√	x	7
LW	√	√	√	√	√	√	x	x	√	√	8
ADDI	√	√	√	√	√	√	x	x	x	√	7

(Resource usage, M1-a 68: LW uses the most resources in series → decides when PC can be updated.) **T_c = slowest = lw = 925 ps** (P&H pipeline-slide version: 950 ps). Exec time (100×10^9 instrs) = #instr × CPI × T_c = 10^{11}×1×925 ps = **92.5 s**. Single-cycle needs separate IM/DM and 3 adders (PC+4, branch, ALU) — no FU reused in a cycle (non-shared FUs).

R6 Multicycle: Tc, CPI, non-architectural state

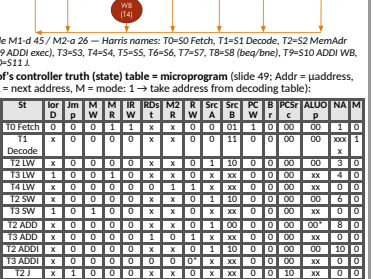
Architectural state (visible to programmer): PC, 32-reg RF, memory. **Non-architectural** (added in multicycle, 5 regs): **IR** (IRWrite), **MDR/Data**, **A**, **B**, **ALUOut** + extra muxes: IorD (shared I/D memory), ALUSrcA (PC/A), ALUSrcB 4:1 (B, 4, SignImm, SignImm<<2), PCSrc 3:1 (ALUResult, ALUOut, JTA). One memory, one ALU (no extra adders) — shared FUs.

$T_c = t_{pcq}+t_{mux}+\max(t_{ALU}+t_{mux}+t_{mem}, t_{mem})+t_{setup}$ = 30+25+250+20 = **325 ps** (prof: each cycle has only **one** ALU op, memory access or RF access; RF faster than memory, memory write faster than read. Path-1: $t_{pcq}+t_{mux}+t_{ALU}+t_{mux}+t_{mux}+t_{setup}$; Path-2: $t_{pcq}+t_{mux}+t_{mem}+t_{setup}$. Slide 65 writes 350 ps — use 325).

Cycles: **lw 5, sw 4, R 4, addi 4, beq 3, j 3**. SPECINT mix 25% lw, 10% sw, 11% branch, 2% j, 52% R: CPI = .25·5+.10·4+.11·3+.02·3+.52·4 = **4.12**. Time = 10^{11}×4.12×325 ps = **133.9 s** (slower than single-cycle! Why: sequencing overhead $t_{pcq}+t_{setup}$ = 30+20 paid every step; but multicycle is less expensive). Prof CPI form: (0.11+0.02)·3 + (0.52+0.10)·4 + 0.25·5 = 4.12. **Homework**: addi, sub, lw, sw, or → 4+4+5+4+4 = 21 cycles, CPI = 21/5 = **4.2**.

Why multicycle (M1-d slides 4–14): single-cycle clock = worst case (lw) for all instrs, unbalanced, redundant adders & 2 memories. Putting a register in the logic: T = Max({src-1,dstn-1},{src-2,dstn-2},...) → equal partitions halve T (f = 2/T). Multicycle = **resource sharing** (FUs, memory, interconnect) + balanced steps; simple instrs finish earlier; controller = FSM giving different signals per step; combine I/D memory, remove redundant adders, add non-architectural regs. PCWrite needed because PC must not change every cycle. **Problems** (slide 4): lw split into 5 steps but T_c doesn't improve 5× (sequencing overhead per step); steps unequal; only one stage busy, others idle; 5 non-arch regs + extra muxes.

R7 Multicycle FSM — prof's T-states (slide 26) = Harris S-states



Slide M1-d 45 / M2-a 26 — Harris names: T0=S0 Fetch, T1=S1 Decode, T2=S2 MemAdr (+S9 ADDI exec), T3=S3, T4=S4, T5=S5, T6=S6, T7=S7, T8=S8 (beq/bne), T9=S10 ADDI WB, T10=S11 J.

Prof's controller truth (state) table = microprogram (slide 49; Addr = µaddress, NA = next address, M = mux address from decoding table):

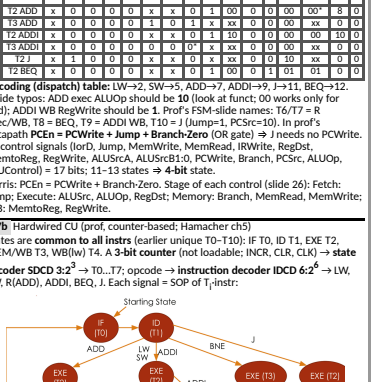
Ad	St	IorD	Jmp	M	W	M R	IR W	RDst	M2 R	R W	Src A	Src B	PC W	PCSr c	ALUOp	NA	M
0	T0 Fetch	0	0	0	1	1	x	x	x	0	0	01	1	0	00	1	0
1	T1 Decode	x	0	0	0	0	x	x	x	0	0	11	0	0	00	xxx 1 x	0
2	T2 LW	x	0	0	0	0	x	x	0	1	1	10	0	0	00	3	0
3	T3 LW	1	0	0	1	0	x	x	0	0	x	xx	0	0	00	xx	4 0
4	T4 LW	x	0	0	0	0	0	1	1	x	xx	0	0	0	00	xx	0 0
5	T2 SW	x	0	0	0	0	x	x	0	1	1	10	0	0	00	6	0
6	T3 SW	1	0	1	0	0	x	x	0	x	xx	0	0	0	00	xx	0 0
7	T2 ADD	x	0	0	0	0	x	x	0	1	1	00	0	0	00	00*	8 0
8	T3 ADD	x	0	0	0	0	1	0	1	x	xx	0	0	0	00	xx	0 0
9	T2 ADDI	x	0	0	0	0	x	x	0	1	1	10	0	0	00	00	10 0
10	T3 ADDI	x	0	0	0	0	0	0	0*	x	xx	0	0	0	00	xx	0 0
11	T2 J	x	1	0	0	0	x	x	0	x	xx	0	0	1	10	xx	0 0
12	T2 BEQ	x	0	0	0	0	x	x	0	1	00	0	1	01	01	0	0

Decoding (dispatch) table: LW→2, SW→5, ADD→7, ADDI→9, J→11, BEQ→12. *Slide typos: ADD exec ALUOp should be **10** (look at funct; 00 works only for add); ADDI WB RegWrite should be **1**. Prof's FSM-slide names: T6/T7 = R exec/WB, T8 = BEQ, T9 = ADDI WB, T10 = J (Jump=1, PCSrc=10). In prof's datapath **PCEn = PCWrite + Jump + Branch·Zero** (OR gate) → J needs no PCWrite. 15 control signals (IorD, Jump, MemWrite, MemRead, IRWrite, RegDst, MemtoReg, RegWrite, ALUSrcA, ALUSrcB1:0, PCWrite, Branch, PCSrc, ALUOp, ALUControl) = 17 bits; 11–13 states → **4-bit** state.

Harris: PCEn = PCWrite + Branch·Zero. Stage of each control (slide 26): Fetch: Jump; Execute: ALUSrc, ALUOp, RegDst; Memory: Branch, MemRead, MemWrite; WB: MemtoReg, RegWrite.

R7b Hardwired CU (prof, counter-based; Hamacher ch5)

States are **common to all instrs** (earlier unique T0–T10): IF T0, ID T1, EXE T2, MEM/WB T3, WB(lw) T4. A **3-bit counter** (not loadable; INCR, CLR, CLK) → **state decoder SDCD 3:2^3** → T0…T7; opcode → **instruction decoder IDCD 6:2^6** → LW, SW, R(ADD), ADDI, BEQ, J. Each signal = SOP of T_i·instr:



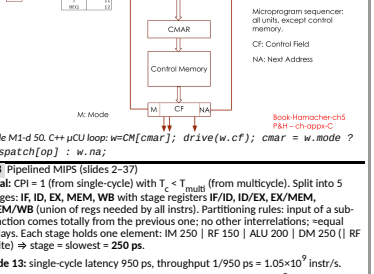
Slides M1-d 52–53 (BNE shown as EXE T3 on slide; same CLR logic).

```
IorD     = T3·(LW+SW)          MemRead  = T0 + T3·LW
IRWrite  = T0                  MemWrite = T3·SW
RegDst   = T3·R                MemtoReg = T4·LW
RegWrite = T3·R + T3·ADDI + T4·LW
ALUSrcA  = T2·(R+LW+SW+ADDI+BEQ)
ALUSrcB1 = T2 + T3·(LW+SW+ADDI)   (11 in T1, 10)
ALUSrcB0 = T0 + T1                (01 in T0, 11)
ALUOp1   = T2·R      ALUOp0 = T2·BEQ
PCWrite  = T0        Jump = T2·J    Branch = T2·BEQ
PCSrc1   = T2·J      PCSrc0 = T2·BEQ
INCR = T0 + T1 + T2·(R+LW+SW+ADDI) + T3·LW
CLR  = T2·(BEQ+J) + T3·(R+SW+ADDI) + T4·LW
```

Counter size: 5 states (T0–T4) → **3 bits**.

R7c Microprogrammed CU (prof slide 50)

Organization: **IR** (opcode) → **decoding table** (external address) → **MUX** (sel = Mode M: 1 = external/dispatch, 0 = NA) → **CMAR** (µPC) → **control memory** word = [**M** | **CF** | **NA**] → CF drives the 15 signals (17 bits). Microprogram sequencer = all units except control memory. Word width = 1 + 17 + 4 = **22 bits**, 13 words. Reduce NA bits: sequence with CMAR+1 (incrementer) and keep only a 2-bit next-type field (next / dispatch / fetch) instead of a 4-bit NA.



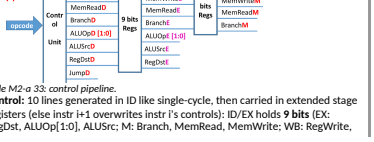
Slide M1-d 50. C++ µCU loop: `w=CM[cmar]; drive(w.cf); cmar = w.mode ? dispatch[op] : w.na;`

R8 Pipelined MIPS (slides 2–37)

Goal: CPI = 1 (from single-cycle) with T_c < T_{multi} (from multicycle). Split into 5 stages: **IF, ID, EX, MEM, WB** with stage registers **IF/ID, ID/EX, EX/MEM, MEM/WB** (union of regs needed by all instrs). Partitioning rules: input of a sub-function comes totally from the previous one; no other interrelations; =equal delays. Each stage holds one element: IM 250 | RF 150 | ALU 200 | DM 250 (|| RF write) → stage = slowest = **250 ps**.

Slide 13: single-cycle latency 950 ps, throughput 1/950 ps = 1.05×10^9 instr/s.

Pipelined: latency 5×250 = 1250 ps, throughput 1/250 ps = 4×10^9 instr/s. Latency unchanged/longer, throughput ideally ×5 (= #stages). n instrs on k stages: (k + n − 1) cycles; speedup = $n \cdot T_{single}/((k+n-1) \cdot T_{stage})$ → k (ideal, balanced).



Slide M2-a 33: control pipeline.

Control: 10 lines generated in ID like single-cycle, then carried in extended stage registers (else instr i+1 overwrites instr i's controls): ID/EX holds **9 bits** (EX: RegDst, ALUOp[1:0], ALUSrc; M: Branch, MemRead, MemWrite; WB: RegWrite, MemtoReg), EX/MEM **5 bits** (M+WB), MEM/WB **2 bits** (WB). Jump used in IF/ID. Strategy-1: pass the instruction through every stage register and decode a local control in each stage. Strategy-2 (P&H): decode once, pass WB/M/EX control fields. Prof: **RF write @posedge, stage regs write @negedge** (so write-then-read in same cycle).

Harris pipelined T_c (branch in ID, fwd): max{ $t_{pcq}+t_{mem}+t_{setup}$ (F); $2(t_{RFread}+t_{mux}+t_{eq}+t_{AND}+t_{mux}+t_{setup})$ (D); $t_{pcq}+t_{mux}+t_{mux}+t_{ALU}+t_{setup}$ (E); $t_{pcq}+t_{memwrite}+t_{setup}$ (M); $2(t_{pcq}+t_{mux}+t_{RFwrite})$ (W) } = 2(150+25+40+15+25+20) = **550 ps**; CPI=1.15 → 10^{11}×1.15×550 ps = **63.3 s**.

	Single	Multi	Pipeline
T_c	925	325	550 (250 ideal)
CPI	1	4.12	1.15 (1 ideal)
Time	92.5 s	133.9 s	63.3 s
FUs	non-shared	shared	shared by several instrs

Hazards: structural (resource), data (RAW), control (branch). **Forwarding:** if (rsE≠0 & rsE==WriteRegM & RegWriteM) ForwardAE=10; else if (rsE≠0 & rsE==WriteRegW & RegWriteW) ForwardAE=01; else 00 (same for rtE→ForwardBE). **lw stall:** lwstall = MemtoRegE·((rsD==rtE)+(rtD==rtE)); Stall=StallD=FlushE=lwstall. **Branch in ID:** ForwardAD = (rsD≠0) (rsD==WriteRegM)RegWriteM; branchstall = BranchD·RegWriteE·(WriteRegE==rsD + WriteRegE==rtD) + BranchD·MemtoRegM·(WriteRegM==rsD + WriteRegM==rtD); FlushD = PCSrcD. Without forwarding: RAW needs 2 NOPs (write 1st half/read 2nd half); P&H branch resolved in MEM → 3-cycle penalty.

R9 IPC > 1: superscalar vs VLIW (slides 38–47)

CPI = cycles/instr; IPC = 1/CPI; time = #instr × (1/IPC) × T. **Static 2-issue (VLIW)**: 64-bit issue packet = ALU/branch + load/store; compiler pairs them, fills with **noop**, may remove all hazards. Adds: 32 more bits from IM, 2 more RF read ports + 1 write port, extra adder for address. **Dynamic (superscalar)**: hardware issues 2–3 instrs/cycle; in-order issue → reservation stations → out-of-order execute (FUs) → in-order commit; scoreboard, Tomasulo. Differences: general vs special instructions; dynamic vs static scheduling; HW vs compiler hazard detection; instruction format; VLIW needs recompilation for each VLIW machine.

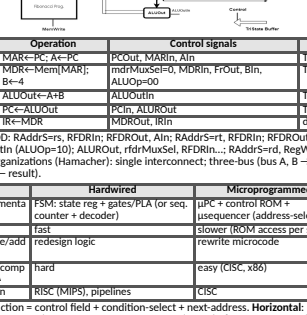
Interconnect \ FUs	Less FUs	More FUs
Less (single bus)	multi-cycle, IPC<1	multi-cycle, IPC<1
More (many buses)	multi-cycle, IPC<1	single-cycle / pipeline, IPC=1

ISA/processor design steps (slide 48): 1 instructions for the algorithm; 2 choose strategy (shared bus / single / multi / pipeline) & design datapath per instr; 3 combine datapaths (insert muxes); 4 clock period from critical path (+setup, clk-to-Q; satisfy hold); 5 identify control signals; 6 design CU (hardwired or microprammed); 7 test & verify.

R10 Shared bus; hardwired vs microprammed

Shared bus: one common set of wires connecting all registers, ALU and memory. Every source drives the bus through a **tri-state buffer** (output enable, e.g. PCBUS, DRBUS); only **one** source may be enabled per cycle (else bus contention); destinations latch with LD signals. → one transfer per bus per cycle, so an instruction takes several cycles (multi-cycle, IPC<1) but needs little interconnect/hardware. Simple CPU fetch: FETCH1 AR←PC; FETCH2 DR←M, PC←PC+1; FETCH3 IR←DR[op], AR←DR[addr]; then execute states (e.g. LD: DR←M; AC←DR). Control by state decoder (I_x·T_i) and OR-ing state terms for each signal.

Prof's design with interconnect constraint (M1-d slides 69–73): 32-bit MIPS for LW, SW, ADDI, ADD, BNE (Fibonacci prog.) with a **one-read-port RF** and a single **SharedBUS** ⇒ multi-cycle (T0–T29 states). RAddrS mux picks rs/rt/rd; RFDR holds RF data; every register has xxIn (load) and xxOut (tri-state onto bus).


One Read Port
RegisterFile
RegWrite
SignExtn
ZeroExtn
ALUOut
ALUIn
PCIn
PCOut
PCIN
PCOUT
PC
4
MUX
RFDR
SEXOUT
ZEXOUT
ALUOUT
ALUIN
ALUCONTROL
ALUSRCA
ALUSRCB
ALUOP
ALUSRCB_1,0
ALUSRCA
ALUCONTROL
ALUOP
ALUSRCB
ALUOUT
ALUIN
PCIN
PCOUT
PCIN
PCOUT
RFDR
RFDROUT
RFDRIN
RFDROUT
RFDRIN
RFDROUT
RFDRIN